ZenoEnergy: Verifier-Preserving Learned Candidate Ordering for UPBA v2 Settlement Search

Dana Edwards

May 18, 2026

Abstract

ZenoEnergy studies whether a small learned energy scorer can reduce candidate search cost for UPBA v2 partial-fill exact-in settlement search while preserving deterministic verification. The system follows the rule: model proposes, verifier decides. A 97-parameter CPU-only linear energy ranker is trained on synthetic, verifier-labeled bounded candidate sets. On a held-out synthetic corpus with 1,983 winner-bearing batches and approximately 20 candidates per batch, the current gap-weighted checkpoint places the exact verifier winner first in 98.34 percent of batches and within the top 2 in 100.0 percent of batches. Mean winner position falls from 19.995 under exhaustive ordering to 1.0166 under learned ordering, while invalid accepts remain zero. The paper records the empirical result, the formal safety boundary, the blockers to production readiness, and a transfer path for applying the same advisory-energy pattern to AutoTrader candidate planning. The current production-adjacent path is an evidence bundle rather than a release claim: source-manifested and coverage-profiled real replay reports are assembled and passed through a fail-closed advisory ranking promotion gate. The latest suffix-bound stress receipt records learned and hybrid mean verifier calls of 1.0132 across a 3-seed by 3-candidate-count bounded synthetic grid, with zero invalid accepts. An adversarial suffix stress also records that declared-output-only suffix bounds fail on injected high-output invalid candidates, while deterministic disqualifiers preserve the certificate.

Keywords: energy-based ranking, deterministic verification, DeFi settlement, candidate search, bounded optimality, UPBA, AutoTrader, advisory machine learning.

1 Introduction

ZenoDEX settlement search is a finite candidate-selection problem once a pool, intent set, price grid, and fill-vector family have been fixed. The verifier can check a submitted certificate deterministically, yet exhaustive candidate ordering may waste verifier calls when the winning candidate appears late in the candidate list. ZenoEnergy v0 asks a narrow empirical question:

> Can a small learned energy scorer reduce UPBA v2 candidate-search cost while leaving deterministic certificate verification unchanged?

The answer on the current bounded synthetic benchmark is affirmative. The learned score is useful because it moves high-quality candidates earlier in the verification schedule. It does not establish settlement validity, candidate set completeness, bounded-grid optimality, or permission to stop early without a deterministic suffix certificate.

2 Problem Statement

For a settlement context x, let $C(x)$ be a finite candidate set, let $V_x(c)$ be the deterministic verifier result for candidate c, and let $J_x(c)$ be the deterministic settlement objective ordered lexicographically by executed volume and surplus. The exhaustive verifier winner is

$$c^\star(x) := \arg\max\{J_x(c) \mid c \in C(x),\ V_x(c) = \text{accept}\}.$$

ZenoEnergy learns a scalar energy

$$E_\theta(x, c) \to \mathbb{R}.$$

Lower energy means earlier verification priority. The optimization target is the expected position of $c^\star(x)$ after sorting by energy:

$$\min_\theta\ \mathbb{E}_x[\text{position}_{\text{sort}(E_\theta)}(c^\star(x))].$$

The settlement rule is unchanged:

$$\text{accepted}(c) := V_x(c).$$

3 Safety Boundary

The implementation keeps the energy module outside the consensus-critical trusted computing base. The dependency direction is one-way:

$$\texttt{src.energy} \to \texttt{src.core.uniform\_batch\_clearing}.$$

Verifier modules do not import `src.energy`. Energy code may import verifier APIs for offline labeling, benchmark construction, and tests.

The core safety property is:

$$\text{LowEnergy}_\theta(x, c) \wedge V_x(c) = \text{reject} \Rightarrow \text{SettlementRejected}.$$

Thus an attractive model score can change order only. It cannot authorize a settlement, mutate ledger state, enter a state root, modify replay, or replace certificate verification.

4 Relation to Energy-Based Reasoning

ZenoEnergy follows the structured-output energy-based learning lineage developed by LeCun and collaborators. In that formulation, an EBM assigns a scalar energy to configurations of observed and predicted variables; inference searches for low-energy configurations, and training shapes the energy surface so desired configurations have lower energy than undesired configurations [2]. LeCun and Huang also frame discriminative EBM training around loss functions that can compare candidate configurations without requiring the full probabilistic normalization cost [1].

Song and Kingma describe classical energy-based models as non-normalized models whose probability density or mass is specified up to an unknown normalizing constant, which makes likelihood training difficult and motivates alternatives such as score matching and noise-contrastive estimation [4]. ZenoEnergy adopts the discriminative, structured-output part of this lineage. It uses an energy as a ranking function:

$$E_\theta(\text{context}, \text{candidate}) \to \text{scalar}.$$

This is closer to the reasoning-energy framing in which an energy score is used to evaluate partial states, constraints, objectives, and progress during planning [5]. It is also compatible with LeCun's broader view of EBMs as scalar compatibility functions for predictive world models and planning under uncertainty [3]. In ZenoEnergy, the partial state is a candidate settlement, the constraints are verifier obligations, and the objective is early discovery of the deterministic winner.

5 Model and Training

The current default model is a linear ranker with 96 feature weights and one bias:

$$E_\theta(x, c) = w^\top \phi(x, c) + b.$$

It has 97 parameters and runs without PyTorch. The optional MLP shape is $96 \to 64 \to 1$, with 6,273 parameters, but the measured artifact in this paper is the no-dependency linear checkpoint.

Training uses pairwise hinge ranking over candidates from the same batch:

$$L = \max(0,\ m + E_\theta(\text{good}) - E_\theta(\text{bad})).$$

The preferred checkpoint uses weighted pair updates:

$$\begin{aligned}\text{pairWeight} = {} & \mathbf{1}_{\text{winnerPair}}\lambda_w + \lambda_v \cdot \text{normalizedVolumeGap} \\ & + \lambda_s \cdot \text{normalizedSurplusGapWhenVolumeTies},\end{aligned}$$

clipped to a fixed maximum. This places more training pressure on the remaining hard cases: valid candidates ranked ahead of slightly better valid winners.

6 Synthetic Candidate Generation

The synthetic generator creates single-pool CPMM exact-in UPBA v2 batches. It samples reserves, fees, users, balances, exact-in intents, candidate price ratios, and deterministic fill vectors. Candidate families include valid candidates, suboptimal valid candidates, balance failures, limit-price violations, negative-reserve cases, invariant failures, noncanonical fill vectors, all-zero candidates, random noisy candidates, attractive output mismatches, unreduced price ratios, and schema/policy mismatches.

Every candidate is labeled by deterministic verifier execution. Each row stores the 96-dimensional feature vector, candidate hash, generator provenance, verifier validity, objective volume, objective surplus, hand energy, target energy, and winner label.

7 Formal Contracts

The Lean development records three layers of safety.

Exact generation. Generated data supports a mathematical optimality claim only when the candidate corpus is sound and complete for the bounded family:

$$\text{GeneratedCorpusExact}(G, F) := \text{CompleteAuditSet}(G, F) \wedge \text{SoundAuditSet}(G, F).$$

Advisory reordering. If ranked order is a permutation of an exact candidate set, a deterministic verifier certificate over the reordered list proves the same bounded global weak optimum. The corresponding Lean theorem is recorded in `UniformBatchOptimality.lean` as the advisory-reordering certificate theorem.

Full fallback. Full fallback is exhaustive-equivalent when the checked order is a permutation of the original finite candidate set:

$$\text{FullFallbackEquivalentOrder}(C, O) := O.\text{Perm}(C).$$

The theorem `full_fallback_equivalent_order_preserves_weak_optimality_iff` states:

$$\text{WeaklyOptimalIn}(w, O) \iff \text{WeaklyOptimalIn}(w, C).$$

Early stop. Stopping before full fallback requires more than a low energy score. The Lean definition `CheckedStopCertificate` requires the current winner to be in the checked set and to weakly dominate both checked candidates and the unchecked suffix. The exact-full checked-stop theorem lifts that certificate to global weak optimality when the full candidate set is exact and the checked prefix plus suffix is a permutation of the full list.

8 Experimental Results

The stored training corpus contains 199,860 candidate rows from 10,000 synthetic batches. The holdout corpus contains 39,979 candidate rows from 2,000 synthetic batches, with 1,983 batches containing at least one verifier-valid candidate. Metrics below are computed over those 1,983 winner-bearing batches.

Table 1: Holdout candidate-ordering performance.

Mode	Top-1	Top-5	Top-10	Mean calls	P99	Invalid accepts
Random	0.045	0.245	0.507	10.467	20	0
Hand energy	0.763	0.996	1.000	1.362	4	0
Learned	0.983	1.000	1.000	1.017	2	0
Hybrid	0.983	1.000	1.000	1.017	2	0

Mean winner position falls from 19.995 under exhaustive order to 1.0166 under learned ordering, a reduction of approximately 94.9 percent. Compared with the hand-coded energy baseline, the learned model reduces mean winner position by approximately 25 percent.

Table 2: Top-k sweep on the holdout corpus. Values are checked-stop audit rates.

Mode	$k = 1$	$k = 2$	$k = 3$	$k = 5$	$k = 10$	$k = 25$
Random	0.045	0.091	0.144	0.245	0.507	1.000
Hand energy	0.763	0.919	0.969	0.996	1.000	1.000
Learned	0.983	1.000	1.000	1.000	1.000	1.000
Hybrid	0.983	1.000	1.000	1.000	1.000	1.000

The checked-stop audit is computed offline after suffix labels are known. It measures whether a deterministic suffix certificate would have justified stopping at that k in the same case.

A later suffix-bound benchmark replaces that offline audit with a deterministic early-stop certificate over the unchecked suffix. A checked verifier winner may stop only when it dominates the checked prefix and every unchecked candidate has a deterministic objective upper bound no better than the winner. On the committed bounded synthetic run with seed 20260541, learned and hybrid

ordering each achieved mean verifier calls of 1.0084, p99 verifier calls of 1, zero invalid accepts, and zero full fallback cases across 119 evaluated batches. Hand energy averaged 1.4202 verifier calls, and random ordering averaged 13.1849.

The cross-seed suffix-bound stress then repeated the benchmark across seeds 20260541, 20260542, and 20260543 with 20, 32, and 50 candidates per batch.

Table 3: Suffix-bound cross-seed stress.

Mode	Configs	Mean calls	P95 max	P99 max	Fallbacks	Invalid accepts
Exhaustive	9	2.3631	5.0000	7.0000	0	0
Hand energy	9	1.3935	4.0000	6.0000	0	0
Learned	9	1.0132	1.0000	4.0000	0	0
Hybrid	9	1.0132	1.0000	4.0000	0	0
Random	9	17.1010	48.0000	50.0000	16	0

Learned and hybrid ordering kept objective-equivalent acceptance, suffix-stop, and certificate-ok rates at 1.0 across all nine configs. This strengthens the bounded synthetic utility claim while leaving the same candidate-family coverage and real replay obligations.

The adversarial suffix stress then injected high-declared-output invalid candidates into the unchecked suffix after the verifier winner was checked.

Table 4: Suffix-bound adversarial stress.

Metric	Value
Evaluated batches	119
Adversary invalid count	119
Adversary disqualified count	119
With-disqualifiers certificate ok	119
Without-disqualifiers certificate ok	0
Declared-output-only forced fail	119

This supports a sharper design lesson: suffix certificates need verifier-derived deterministic invalidity signals. Raw declared outputs are too weak against attractive invalid suffixes.

9 Model Audit

The gap-weighted checkpoint has no forbidden label-like feature names and no nonzero reserved-feature weights. Its largest positive weights are hard verifier-shaped penalties inherited from hand initialization: negative reserves, CPMM invariant failures, limit-price violations, balance violations, malformed fill vectors, schema/policy mismatches, and output mismatches. Its largest negative weights reward executed volume and surplus:

candidate_normalized_executed_volume	−58.0118
candidate_normalized_surplus	−28.7780
candidate_volume_log1p	−9.7421
candidate_surplus_signed	−7.6317.

This supports the intended interpretation: malformed candidates stay expensive, while higher-objective valid-looking candidates move earlier.

10 Why This Is Not Production Ready

ZenoEnergy v0 is research-useful and operationally small, yet several blockers prevent production readiness.

1. **Synthetic bounded evidence.** The current results come from a synthetic generator. This is appropriate for a controlled benchmark, but it does not show distributional fit to real orderflow.
2. **No privacy-approved replay corpus.** A production-adjacent claim needs non-private or properly redacted replay data with verifier-backed labels.
3. **Candidate-family exactness is still a proof obligation.** The synthetic corpus is an experiment. Production needs exact candidate generation or a certificate that the bounded family under discussion is sound and complete.
4. **UPBA v2 bounded-grid optimality evidence is incomplete.** The scorer helps rank v2 candidates, while production settlement claims still need the v2 bounded-grid optimality verifier or an equivalent certificate.
5. **Live early stopping requires production-grade suffix evidence.** Top-k recall is an empirical accelerator. The current suffix-bound certificate is a deterministic prototype over generated finite lists. The adversarial stress shows that declared-output-only bounds are insufficient and that deterministic disqualifiers are part of the certificate design. Production early stop still requires candidate-family coverage, real replay, and source-manifested evidence that the bound remains useful on representative traffic.
6. **Scope is narrow.** The experiment covers a single pool, CPMM, exact-in swaps, fixed admitted intents, and partial fills. Multi-pool routing, exact-out, confidential orderflow, latency races, and adversarial live behavior remain outside the measured surface.
7. **Operational controls are missing.** A production deployment needs model artifact signing, version pinning, rollback policy, drift detection, resource bounds, replay conformance, incident response, and governance review.

These are engineering and assurance blockers. They do not undermine the research result. They define the next release gate.

11 Production Gate and Evidence Bundle

ZenoEnergy now has a production-adjacent evidence path:

`tools/build_zenoenergy_production_evidence_bundle.py`.

The bundle composes five deterministic artifacts:

`zenodex/energy/upba_real_replay_report/v1`,
`zenodex/energy/autotrader_real_shadow_report/v1`,
`zenodex/energy/replay_source_manifest_check/v1`,
`zenodex/energy/replay_coverage_profile_check/v1`,
`zenodex/energy/production_promotion_gate/v1`.

The bundle itself uses schema

`zenodex/energy/production_evidence_bundle/v1`.

Operators build source manifests with

`tools/build_zenoenergy_replay_source_manifest.py`.

That command computes canonical source-report hashes, attaches deterministic replay and no-live-secrets attestations, records the secret-scan result, runs the manifest checker, and writes the manifest only when the check passes. Operators can generate the secret-scan report with

`tools/check_zenoenergy_replay_secret_scan.py`.

The scanner writes

`zenodex/energy/replay_secret_scan/v1`,

catches obvious key material and sensitive JSON keys, and can be supplied to the manifest builder with `-secret-scan-report`. Dirty scans and source-count mismatches fail closed. A clean scan is a packaging guardrail; production promotion still requires the production gate and source-manifested real replay reports.

The coverage-profile checker writes

`zenodex/energy/replay_coverage_profile_check/v1`.

It requires UPBA replay breadth across pools, intent-size buckets, candidate families, hard-negative families, and market days. It requires AutoTrader shadow replay breadth across strategy, guard, and decision families. The profile is a breadth guard against aggregate-count-only promotion; it does not prove representativeness of future traffic.

The release predicate is:

$$\begin{aligned}\text{ProductionEvidenceBundle} := \text{UPBARealReplayReport} \\ \wedge\ \text{AutoTraderRealShadowReport} \\ \wedge\ \text{ReplaySourceManifestChecks} \\ \wedge\ \text{ReplayCoverageProfileChecks} \\ \wedge\ \text{ProductionPromotionGate}.\end{aligned}$$

The production gate requires at least 1,000 real UPBA replay batches, 20,000 real UPBA candidates, 500 real AutoTrader shadow contexts, 5,000 AutoTrader shadow rows, seven market days, top-25 recall at least 0.99, zero invalid accepts, deterministic replay, no live secrets, source manifest checks, and coverage profile checks. It also requires learned mean verifier or guard calls below hand energy.

Malformed evidence fails closed. Well-formed but insufficient evidence produces `decision: blocked`. A passing bundle can only support advisory ranking:

$$\text{LowEnergy}(c) \wedge \text{VerifierRejects}(c) \Rightarrow \text{SettlementRejected}.$$

The current gate remains blocked because the repository contains synthetic and fixture evidence, plus the tooling needed to evaluate real evidence, while the required real replay reports have not yet been supplied.

12 AutoTrader Transfer

The ZenoEnergy pattern applies naturally to AutoTrader if AutoTrader decisions are represented as finite candidate plans. Let $A(s)$ be a candidate action or strategy set for trading state s, let $G_s(a)$ be the deterministic policy, risk, authorization, and budget gate, and let $U_s(a)$ be the deterministic utility or risk-adjusted objective used by the verifier. An advisory scorer can rank candidates:

$$E_\phi(s, a) \to \mathbb{R},$$

while the execution rule remains

$$\text{executable}(a) := G_s(a).$$

This would let AutoTrader use learned energy for ordering candidate strategies, repairing low-quality plans, and reducing evaluation cost. The same safety rules carry over:

- the model ranks strategies but cannot authorize trades;
- deterministic policy gates decide execution;
- model output does not enter balances, positions, state roots, or signed orders;
- full fallback checks a permutation of the original candidate plans;
- early stop requires a deterministic certificate for unexamined candidate plans.

The best first AutoTrader experiment is not live trading. It is a replayed, non-private candidate-plan benchmark with deterministic risk labels. The metric should mirror ZenoEnergy: does learned ordering place the gate-accepted, highest-utility plan in the first few verifier checks without changing any execution predicate?

13 Conclusion

ZenoEnergy v0 provides strong evidence that a tiny energy ranker can reduce candidate-search cost while preserving deterministic settlement verification. The current 97-parameter model moves the winner to position 1.0166 on average and to the top 2 in all held-out winner-bearing batches. The result is powerful because the model is small, interpretable, CPU-only, and isolated from consensus-critical paths.

The release path is clear. Keep the scorer advisory, add real replay evidence, finish exact candidate-generation and v2 optimality certificates, require a deterministic suffix certificate for live early stop, and pass the production evidence bundle gate. The same verifier-preserving ranking architecture can then be tested in AutoTrader candidate planning.

References

[1] Yann LeCun and Fu Jie Huang. *Loss Functions for Discriminative Training of Energy-Based Models*. Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics, PMLR R5:206–213, 2005. https://proceedings.mlr.press/r5/lecun05a.html.

[2] Yann LeCun, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, and Fu Jie Huang. *A Tutorial on Energy-Based Learning*. In G. Bakir, T. Hofmann, B. Schölkopf, A. Smola, and B. Taskar, editors, *Predicting Structured Data*, MIT Press, 2006. https://yann.lecun.org/exdb/publis/pdf/lecun-06.pdf.

[3] Yann LeCun. *A Path Towards Autonomous Machine Intelligence*. Version 0.9.2, 2022. https://openreview.net/pdf/315d43ba26f55357a84cec9a7ed15a6610094f79.pdf.

[4] Yang Song and Diederik P. Kingma. *How to Train Your Energy-Based Models*. arXiv:2101.03288, 2021. https://arxiv.org/abs/2101.03288.

[5] Eve Bodnia and Boris Hanin. *Energy-Based Models for Reasoning, LLMs for the Interface: Scaling Reasoning with Agentic AI*. Logical Intelligence, January 21, 2026. https://logicalintelligence.com/blog/energy-based-models-for-reasoning.

[6] Dana Edwards. *ZenoEnergy v0 Results and Formal Receipts*. Autonomous Tau DEX repository, 2026.